Spectral Composition: A New Paradigm for Sample-Based Music Creation via Latent Diffusion

Lorenzo Moriondo
tunedconsulting@gmail.com
lorenzo@genefold.ai

tuned.org.uk
Genefold AI Ltd

August 15, 2026

https://github.com/tuned-org-uk/latent-sound-diffusion

Abstract

We introduce *Spectral Composition*, a novel method for sample-based music production that treats composition as iterative navigation of a compressed spectral latent manifold rather than direct waveform or symbolic editing. A frozen ArrowSpace graph Laplacian L_F and its associated energy-dispersion network λ^{ED}, estimated once from the producer's entire sample library, define a low-dimensional feature-space manifold over which all generative and compositional operations are performed. The method proceeds in four stages: **(1) spectral encoding** ("dehydration") of the library into a compact spectral representation $(L_F, U_q, \lambda^{\mathrm{ED}})$; **(2) dense unconditional sampling** of a sound bank via a spectrally-conditioned 1-D diffusion transformer (DiT) operating on EnCodec audio latents, decoded through a graph-structured decoder that filters every latent frame through the corpus's frozen spectral eigenbasis; **(3) MIDI-conditioned materialisation** ("rehydration") that pitch-shifts and re-timbres generated sounds according to a language-model-generated MIDI file; and **(4) recursive variant generation** by feeding outputs back through the model with successive MIDI sequences. The resulting family of variants, each sharing spectral lineage with the source library, forms a natural set of stems for multitrack mixing. The paradigm is grounded in two spectral mechanisms: a per-time-step graph-filter decoder and an entropic stopping clock derived from the same eigenstructure. A preliminary-scale, fully local evaluation (two CC-licensed corpora, consumer hardware, minutes to train) attaches a first quantitative trace to each stage — 15.99× corpus-scale compression, waveform-fidelity parity with a matched unconstrained decoder, drift-then-saturate recursive novelty, and up to 4× sampling-compute reduction at equal or better distributional quality — reported as calibrated first traces with open fronts named, not as benchmark claims. Spectral Composition

is built on the Latent Sound Diffusion (LSD) system [1], which is a sound-production fork of ArrowSpace Latent Diffusion with Spectral Chart Conditioning (ALD-SC) [2], and is exposed to the producer via the LSD-Studio desktop application [3].

Contents

1 Introduction

Traditional sample-based music production treats a sample library as a fixed, enumerable set of audio assets: the producer browses, triggers, slices, and layers clips inside a digital audio workstation (DAW), guided by perception and taste. This workflow has scaled to industrial size through large commercial sample libraries, but it fundamentally limits compositional novelty to recombination of existing clips without access to the generative structure of the collection.

Spectral Composition proposes a different model: the sample library is a *corpus* from which a frozen spectral prior is estimated once, after which composition is an act of sampling and recombination inside the resulting latent manifold. This reframes "browsing samples" as "traversing a spectral graph", and "arranging a track" as "recursively rehydrating dense latent draws against symbolic control signals."

Calibration This paper introduces a paradigm and its reference implementation. The evaluation of Section 6 runs at deliberately preliminary scale — 256 clips, 20 training epochs, one consumer GPU — and is calibrated accordingly: each property of the workflow receives a first quantitative trace, negative results are reported verbatim, distributional scores are computed with documented proxies, and every stronger claim is converted into a named experiment for the scale-up. The paper positions itself by categorical distinction: existing audio-generation systems instantiate different design spaces, and no comparison against them is claimed or implied. Terminology is used deliberately: the *paradigm* is the conceptual inversion (novelty as traversal of a fixed manifold); the *workflow* is what the producer executes (dehydrate, sample, rehydrate, recurse); the *pipeline* is the software realisation (LSD model core + LSD-Studio).

1.1 Contributions

- We define the Spectral Composition pipeline: a four-stage method (dehydration, sampling, materialisation, recursion) that operationalises compositional workflows over a spectral latent manifold.
- We show how the existing LSD inference contract (`generate_sound_bank`, `condition_on_audio`, `synthesize_midi`) directly implements stages 2–4 of the pipeline, with the language-model MIDI front-end shipping inside LSD-Studio.
- We articulate the mathematical and architectural foundations of the workflow, instantiating the ALD-SC research programme's central *hypothesis* — that decoding on the feature-space manifold $(L_F, \lambda^{\mathrm{ED}})$ preserves corpus spectral structure under compression better than an unconstrained ambient decoder — and providing first measurements: waveform-fidelity parity or better and per-band energy-retention gains for the chart-constrained decoder, with the distributional gap named explicitly as the scale-up experiment (Section 6.2).
- We describe LSD-Studio as the producer-facing realisation of the pipeline, assigning multitrack mixing and workflow management to the application layer.
- We contribute the evaluation harness itself — reproducible metrics, sweeps, and documented proxy methodologies — that attaches a first quantitative trace to each property

of the workflow and defines the measurements on which the paradigm's stronger claims should later be judged.

1.2 Paper Organisation

Section 2 reviews background in latent diffusion for audio, ArrowSpace graph priors, and entropic time scheduling. Section 3 defines the four stages of Spectral Composition. Section 4 describes the underlying model architecture. Section 5 details the implementation and datasets. Section 6 presents evaluation methodology. Section 7 discusses the method as a creative paradigm. Section 8 outlines future work, and Section 9 concludes.

2 Background

2.1 Latent Diffusion for Audio

Latent diffusion models [8] move the expensive iterative denoising process from pixel (or waveform) space into a compressed continuous latent space produced by a pretrained autoencoder, drastically reducing computation while retaining high-fidelity synthesis. EnCodec [9] provides the audio equivalent: a neural codec operating at 24 kHz that encodes mono audio clips into a 1-D continuous latent $z \in \mathbb{R}^{D\times T}$ (with $D = 128$, $T = 375$ for 5-second clips) via its pre-quantization continuous features.

Beyond this codec-in-latent-space pattern, the field has explored three complementary strategies. *Discrete-token language models* — Jukebox [15], AudioLM [16], MusicLM/ MusicGen [17, 18] — cast generation as next-token prediction over neural codec codes, inheriting the long-range coherence of sequence models but at heavy sampling cost and with identity tied to massive generic corpora rather than to a producer's own archive. *Continuous latent diffusion* — Riffusion [19], AudioLDM 2 [20], Stable Audio [21], Moûsai [22] — denoises in a compressed continuous latent (our setting), trading the discrete hierarchy for sample efficiency and smooth interpolation; these systems are text-conditioned and corpus-wide, designed to synthesise arbitrary described audio. *Symbolic generative models* produce MIDI or scores directly and remain trivially controllable but carry no timbral identity of their own.

Spectral Composition occupies a deliberately different position. All of the above condition generation on a *prompt* and evaluate against a *generic* distribution; none treats a private sound archive as the primary generative substrate. Our latents come from the producer's library via a frozen EnCodec encoder [9]; our conditioning vector c_{spec} is corpus-derived, not text-derived; and our generation contract (sample, rehydrate, recurse) is a compositional workflow rather than a single-conditioning synthesis call. Symbolic structure enters not as the output but as a *control surface* (Stage 3), combining the controllability of the MIDI world with the timbral lineage of the latent-diffusion world.

2.2 ArrowSpace Graph Priors and Energy-Dispersion Networks

ArrowSpace [6] defines a spectral search framework for embeddings in which feature columns of a corpus feature matrix $A \in \mathbb{R}^{N\times F}$ are treated as nodes of a feature-space graph $G_F = (V_F, W_F)$ with $|V_F| = F$. The feature-space graph Laplacian

$$L_F = D^{-1/2}(D - W)D^{-1/2}, \qquad L_F = U\Lambda U^{\top}, \tag{1}$$

has eigenvectors $U_q \in \mathbb{R}^{F\times q}$ (the q smoothest modes) that define the semantic subspace of the corpus.

The Energy Dispersion Network (EDN) [7] associates to each feature f a per-feature energy-dispersion scalar $\lambda_f^{\mathrm{ED}} \in [0,1]$, encoding how semantic structure is distributed over the feature graph. Projected onto the chart, these give per-mode dispersion weights

$$\lambda_k^{\mathrm{chart}} = \sum_{f=1}^{F} \lambda_f^{\mathrm{ED}}\, U_{f,k}^2. \tag{2}$$

In ArrowSpace's dual-space construction the corpus is viewed through two graphs: an *item graph* over the N clips and a *feature graph* over the F feature dimensions, where each feature is represented by its signal across the corpus (the corresponding column of A). Edges of the feature graph are k-nearest-neighbour links under cosine similarity, symmetrised to keep L_F well-defined. The spectral theorem then orders features by *smoothness*: the leading eigenvectors U_q span the subspace along which corpus energy varies most coherently — empirically the directions that encode style and timbral family rather than idiosyncratic detail. This is the property the ArrowSpace index exploits at query time via $\lambda\tau$-modulation, weighting chart coordinates by dispersion to rank spectrally-near neighbours.

The Energy Dispersion Network completes the picture by diagnosing *how concentrated* the semantic structure of a corpus is: λ_f^{ED} large on a tightly-connected neighbourhood of the feature graph means the corpus's meaning is carried by a compact, redundant feature set — the regime in which aggressive compression (small q) preserves retrieval behaviour. The connection we draw on in this paper is direct: $\lambda_k^{\mathrm{chart}}$ (Eq. 2) measures how much corpus-level dispersion each smooth mode carries, and we use it to gate how much reconstruction energy the decoder allocates to that mode. Semantic coherence under compression is thus not an after-the-fact evaluation but a structural input to decoding.

2.3 Entropic Time Scheduling and the Barontini Clock

The entropic (Barontini) clock [10, 11] provides an intrinsic, spectrally-informed stopping criterion for diffusion sampling. Per-mode entropy-time coordinates are defined as

$$\tau_k(t) = \nu_k \bar{\alpha}_k(t), \tag{3}$$

where ν_k are the eigenvalues of L_F and $\bar{\alpha}_k(t)$ is the mode-k noise schedule. The sampler terminates when

$$\sum_{k=1}^{q} \nu_k\, \bar{\alpha}_k(t) < \varepsilon, \tag{4}$$

the "heat-death" criterion, replacing an arbitrary fixed step count with a spectrally-motivated halting rule [1, 2].

The physical motivation is the *problem of time* in closed systems. Connes and Rovelli [23] argue that in a generally covariant theory the physical time-flow need not be a universal external parameter but can arise from the thermodynamical state of the system itself. Barontini's cold-atom experiment [10] provides an operational testbed: a closed system partitioned into observed and unobserved sectors admits an *entropic time* constructed purely from internal entropy exchange between the sectors, without any external clock. The heat-death criterion above is the sampling-time analogue: the sampler's progress is

measured by how much spectral structure remains unresolved (internal state), and halting is a property of the trajectory, not of a step counter.

Among adaptive-sampling methods, DDIM [24] and DPM-Solver [25] accelerate diffusion by reusing or integrating the same fixed schedule more efficiently — the clock is external and trajectory-independent. The entropic stopping rule is complementary rather than competing: it says *when individual modes have nothing left to resolve*, per-mode, driven by the corpus eigenvalues ν_k. In our implementation the criterion is evaluated as the remaining dissipation $\sum_k \nu_k(1-\bar{\alpha}_k(t)) < \varepsilon$, which decreases monotonically as denoising completes (Section 6.3 measures the resulting steps-vs-quality tradeoff). We retain this as a design principle — generative dynamics informed by state-derived geometry — while acknowledging that the full thermodynamic construction remains a metaphor-level grounding at this stage.

3 Spectral Composition

We now define the four stages of the Spectral Composition pipeline formally. Let $\mathcal{X} = \{x_1, \ldots, x_N\}$ denote the producer's sound archive (a set of raw audio clips), and let $\mathcal{M}$ denote a trained LSD model (frozen prior + graph decoder + 1-D DiT).

3.1 Stage 1 — Spectral Encoding (Library Dehydration)

3.1.1 EnCodec feature extraction

Each clip $x_i \in \mathcal{X}$ is encoded by the frozen EnCodec encoder E_ϕ (24 kHz mono, never updated) to produce:

$$(z_i,\ A_i) = E_\phi(x_i), \qquad z_i \in \mathbb{R}^{D\times T}, \quad A_i = \mathrm{Pool}(z_i) \in \mathbb{R}^F. \tag{5}$$

The pooled feature field A_i is the row contributed by clip i to the corpus feature matrix $A \in \mathbb{R}^{N\times F}$.

3.1.2 ArrowSpace prior construction

From A the ArrowSpace adapter builds the frozen feature-space graph Laplacian L_F and energy-dispersion network λ^{ED} via the `pyarrowspace` library [5]. The eigenvectors U_q and eigenvalues ν_k are stored as non-trainable buffers. The spectral conditioning vector for a clip is

$$c_{\mathrm{spec}} = \left[\tilde{e},\ \lambda^{\mathrm{chart}},\ \nu\right] \in \mathbb{R}^{3q}, \tag{6}$$

where $\tilde{e}_k$ is the per-mode band energy. The entire library is now represented by the compact frozen objects $(L_F, U_q, \lambda^{\mathrm{ED}})$ — the *dehydrated* spectral representation.

Dehydration as compression The dehydration step is also a compression statement. Representing the library as raw 16-bit PCM costs $N \cdot T \cdot 16$ bits; the dehydrated representation costs the prior (L_F, U_q, ν, λ^{ED}, λ^{chart} — 2.2×10^5 floats, stored once, corpus-independent of N) plus per-clip storage at EnCodec's 24 kbps code rate. Table 1 reports the resulting ratios for our evaluation corpus ($T = 96{,}000$ samples, 4 s at 24 kHz): the prior cost dominates a single clip (2.34×) but amortises rapidly, reaching 15.99× at the corpus scale of 9,811 NSynth notes. The asymptote is set by EnCodec's code rate

alone; the spectral prior adds a constant, shrinking relative overhead. Note what this compression buys: not just a smaller library, but a *generative* representation — the same $(L_F, U_q, \lambda^{\text{ED}})$ that stores the corpus also constrains every sampling and decoding operation in Stages 2–4.

Table 1: Dehydration compression ratio vs. corpus size N (4 s clips, 16-bit PCM vs. frozen prior + 24 kbps per-clip code rate; prior is 561,664 bits stored once).

N	Compression ratio
1	2.34×
10	10.09×
50	14.32×
100	15.12×
256	15.64×
1000	15.91×
9811	**15.99**×

3.1.3 Two-stage model training

Training proceeds as two frozen-then-fine-tuned phases. In Phase 1 the graph decoder is trained with optional latent-space noise injection (parameter `NOISE_INJECT` $\in [0, \infty)$). In Phase 2 the 1-D DiT denoiser is trained on the corpus latents $\{z_i\}$ conditioned on c_{spec}:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0,\epsilon,t} \left\| v - v_\theta(z_t,\, t,\, c_{\text{spec}}) \right\|_2^2 , \tag{7}$$

using v-prediction for numerical stability.

The full training objective for Phase 1 combines four terms:

$$\mathcal{L} = \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_{\text{stft}}\mathcal{L}_{\text{stft}} + \lambda_{\text{chart}}\mathcal{L}_{\text{chart}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}}, \tag{8}$$

with $\mathcal{L}_{\text{rec}}$ the waveform L1 distance, $\mathcal{L}_{\text{stft}}$ a multi-scale spectral loss (spectral convergence + log-magnitude L1 over FFT sizes 512/1024/2048), $\mathcal{L}_{\text{chart}} = \|\tilde{e}(A) - \tilde{e}(\hat{A})\|^2$ the spectral-chart consistency between the band energies of encoder features and their reconstruction, and $\mathcal{L}_{\text{smooth}} = \|A(I - U_q U_q^\top)\|_F^2 / \|A\|_F^2$ the off-manifold penalty. Two implementation facts matter for interpreting our results, and we state them plainly. First, in the configuration evaluated in Section 6 we set $\lambda_{\text{stft}} = 0$, so decoder training is driven by the L1 term (with $\mathcal{L}_{\text{chart}}, \mathcal{L}_{\text{smooth}}$ logged as corpus-level diagnostics; in the current trainer the chart target is detached, so they contribute no decoder gradient). Second, the chart term as specified — comparing decoded audio re-encoded to $\hat{A}$ against A — is the natural form of the objective and is measured as such at evaluation time (Table 6); wiring it into the decoder gradient is scheduled future work.

The *non-repeatable training* design is deliberate. With `SEED=None` a fresh seed is drawn per run, so every baked model — and therefore every bank and variant family — is a singular artefact: two producers dehydrating the same library obtain different-but-lineage-identical manifolds. `NOISE_INJECT` (σ_{inj}) perturbs the latent the decoder sees during training (the conditioning c_{spec} stays derived from the clean features), trading reconstruction fidelity for acoustic variety (Section 6.3 quantifies the tradeoff). Training uses Adam with global gradient-norm clipping at 1.0; a non-finite loss or gradient norm aborts the run with an explicit error, a hardening added after diagnosing an early gradient pathology in the wave block (Section 4.3).

3.2 Stage 2 — Dense Unconditional Sampling (Sound-Bank Generation)

Given trained $\mathcal{M}$, a sound bank $\mathcal{B}$ of n clips is generated by running the DDIM or Euler sampler on the 1-D DiT unconditionally, then decoding through the `ClockGatedGraphDecoder`:

$$\hat{z}^{(j)} \sim p_\theta(z \mid c_{\text{spec}}), \qquad j = 1, \ldots, n, \tag{9}$$

$$\hat{x}^{(j)} = D_\psi\big(\hat{z}^{(j)};\ U_q,\ \lambda^{\text{ED}},\ c_{\text{spec}}^{(j)}\big)\,, \tag{10}$$

where $c_{\text{spec}}^{(j)}$ is derived *self-consistently* from $\hat{z}^{(j)}$ itself (no separate chart sampling required). The bank $\mathcal{B} = \{\hat{x}^{(1)}, \ldots, \hat{x}^{(n)}\}$ consists of samples that are perceptually self-similar — drawn from the same frozen manifold — yet acoustically distinct because `SEED=None` is the default.

The bank is *super-dense* in the sense that every draw lands on the same frozen manifold yet no two draws coincide. Three mechanisms conspire. (i) The prior is corpus-level, not per-clip: U_q and λ^{ED} encode what the *whole library* has in common — its spectral signature — so every decoded clip is filtered through the same stylistic identity. (ii) The unconditional DiT is trained on latents of the library only; its stationary distribution is the library's latent distribution, so dense sampling sweeps that distribution rather than an open-ended audio space. (iii) The self-consistent chart conditioning keeps each draw within the energy allocation the corpus actually exhibits. Locally, draws differ because the DiT's stationary measure is continuous and `SEED=None` makes the sampling trajectory fresh each time; globally, they agree because every path through Stage 2 crosses the same projector Π_q. In producer terms: one bank, many voices, one sound. This is precisely the behaviour a sample-library workflow wants — a coherent palette rather than a bag of unrelated one-shots — and it is a structural consequence of the frozen-prior design, not a tuned temperature.

3.3 Stage 3 — MIDI-Conditioned Materialisation (Rehydration)

3.3.1 Language-model MIDI generation

A language model (e.g. a GPT-based symbolic music model) generates a MIDI file $\mathcal{P} = \{(p_i, t_i, d_i)\}$ where p_i is the MIDI pitch, t_i the onset time, and d_i the duration of note i.

Implementation status. This front-end is implemented in the shipped system: LSD-Studio carries a proprietary symbolic-generation worker that emits quantised event lists conforming to the (p_i, t_i, d_i) schema `synthesize_midi` consumes, with producer controls over musical role (melodic, chordal, bass, percussive), length, and note density. Its design is undisclosed; what matters for the pipeline is the interface, which is deliberately swappable: any source of symbolic material — hand-played MIDI, rule-based generators, imported MusicXML rendered to MIDI, or clip detection from the archive itself — feeds Stage 3 unchanged, because the materialisation interface is the event list, not the generator. The evaluation of Section 6 uses fixed scores for exactly this reason (reproducibility), and the front-end is treated as out of scope for the model-level measurements.

3.3.2 Pitch-shifted decoding (synthesize_midi)

For each note (p_i, t_i, d_i) the adapter selects a bank clip $\hat{x}^{(j)}$ and applies pitch-shifting to MIDI pitch p_i via the `synthesize_midi` method of `LSDModel`:

$$y_i = \text{PitchShift}\big(\hat{x}^{(j)},\ \Delta p_i\big)\Big|_{[t_i,\ t_i+d_i]}, \tag{11}$$

where Δp_i is the semitone offset from the bank clip's reference pitch. The materialised output is

$$Y^{(1)} = \sum_i y_i, \tag{12}$$

a waveform that follows the symbolic structure of $\mathcal{P}$ while carrying the spectral identity of the source library.

3.3.3 Audio-conditioned variant generation

Optionally, the materialised output $Y^{(1)}$ is fed back into the model via `condition_on_audio` (Mode B) to generate variants:

$$\hat{z}' \sim p_\theta(z \mid z_{\text{cond}} + \eta), \qquad \eta \sim \mathcal{N}(0, \sigma^2 I), \tag{13}$$

where $z_{\text{cond}} = E_\phi(Y^{(1)})$ is the conditioning latent and σ is the perturbation strength. The chart conditioning c_{spec} is re-derived from $\hat{z}'$, keeping the output on the archive manifold.

3.4 Stage 4 — Recursive Variant Generation

Stages 2 and 3 are applied recursively. Let $\mathcal{P}^{(1)}, \ldots, \mathcal{P}^{(R)}$ be R distinct MIDI files (generated by the language model or supplied by the producer). The recursive scheme is:

$$\mathcal{B}^{(0)} = \texttt{generate\_sound\_bank}(n), \tag{14}$$

$$Y^{(r)} = \texttt{synthesize\_midi}\big(\mathcal{P}^{(r)},\ \mathcal{B}^{(r-1)}\big), \tag{15}$$

$$\mathcal{B}^{(r)} = \texttt{condition\_on\_audio}\big(Y^{(r)},\ n,\ \sigma_r\big), \qquad r = 1, \ldots, R. \tag{16}$$

Each round deepens the stylistic imprint of the source library while incorporating new symbolic content from $\mathcal{P}^{(r)}$. The family $\{Y^{(1)}, \ldots, Y^{(R)}\}$ constitutes a set of *variants* related by shared spectral ancestry.

3.4.1 Convergence and heat-death stopping

The recursive process is bounded intrinsically by the heat-death criterion (Eq. 4): if the conditioning latent z_{cond} has collapsed to a near-constant spectral chart, additional conditioning passes yield diminishing novelty. In practice, the producer controls depth via R and the perturbation schedule $\{\sigma_r\}$.

3.5 Final Assembly — Multitrack Mixing

The variant outputs $\{Y^{(1)}, \ldots, Y^{(R)}\}$ plus any selections from the unconditional bank $\mathcal{B}^{(0)}$ form a natural stem set for a multitrack mix. In the shipped system this surface is provided by LSD-Studio: each generated process occupies a channel strip with mute/solo

and level controls under a master fader, and samples can be sculpted with an offline ADSR envelope before auditioning or export. Panning, insert effects, and DAW bridging are scheduled extensions; in the meantime, long-form post-processing (overlap-and-add generation, BPS-synchronised modulation, latent-space concatenation) is available via the notebook series (02–04) [1].

4 System Architecture

4.1 Frozen ArrowSpace Prior and Manifold Construction

The ArrowSpace prior is built once from the corpus by `build_prior.py` and stored as non-trainable buffers:

$$\Pi_q = U_q U_q^\top \in \mathbb{R}^{F\times F}, \tag{17}$$

the spectral projector onto the smooth subspace. The frozen design constraint means that L_F and U_q never receive gradient updates; only the graph decoder and DiT are trained. This ensures that the manifold geometry is a property of the *corpus*, not a function of any single training run.

Concretely, `build_prior.py` constructs the graph as follows. Feature signals (columns of the corpus matrix $A \in \mathbb{R}^{N\times F}$, F=128 for EnCodec features) are ℓ_2-normalised and compared by cosine similarity; each feature is linked to its k nearest neighbours (k=4 in the reported configuration), and the adjacency is symmetrised as $W = \max(W, W^\top)$ so the Laplacian is well-defined. The unnormalised Laplacian $L_F = D - W$ is eigendecomposed once (`torch.linalg.eigh`), eigenvalues sorted ascending, the trivial constant mode dropped, and the leading q eigenvectors retained as U_q. The dispersion network is the normalised per-feature energy $\lambda_f^{\mathrm{ED}} = \|A_{\cdot f}\|_2^2 / \|A\|_F^2$, projected onto the chart to yield λ^{chart}. All five objects are frozen buffers with zero trainable parameters.

The `wire_graph.py` adapter is the port to the full ArrowSpace library: when the `pyarrowspace` Rust bindings are importable (exposing the graph constructor `gl` and dispersion query `lambdas()`), L_F and λ^{ED} are computed by the library — the single source of truth, including its sparse Laplacian normalisation — and the Python kNN construction above serves as the *fallback* used when the bindings are unavailable (CI, platforms without the Rust toolchain). The evaluation reported here ran the fallback path; the two constructions agree on ordering of the leading modes, and the fallback's dense 128×128 Laplacian is negligible compute at this scale. For archives where F grows with the feature extractor, the library's sparse implementation is the intended path (Section 6.4).

4.2 1-D DiT Denoiser

The denoiser `MinimalDiT` patchifies the 1-D latent $z \in \mathbb{R}^{D\times T}$ with stride-s convolutions, applies L transformer blocks with AdaLN conditioning on (t, c_{spec}), and unpatchifies to predict velocity v. Classifier-free guidance (CFG) is applied by randomly dropping c_{spec} during training with probability p_{drop}, enabling guidance-scale control at inference.

4.3 Graph Decoder

The `ClockGatedGraphDecoder` processes the denoised latent $\hat{z}$ through a sequence of `WaveReconstructionBlocks`, each applying a *per-time-step graph filter* over the activation

field. With activations $H \in \mathbb{R}^{C\times T}$ and feature projections $A = W_{C\to F}H \in \mathbb{R}^{F\times T}$:

$$\hat{H} = U_q^\top A \in \mathbb{R}^{q\times T}, \tag{18}$$

$$g = \sigma\big(W_g\, c_{\text{spec}}\big) \in \mathbb{R}^{q}, \tag{19}$$

$$A' = U_q\big(\hat{H} \odot g\mathbf{1}^\top\big) \in \mathbb{R}^{F\times T}, \tag{20}$$

followed by $H' = H + W_{F\to C}A'$ and a residual `GroupNorm--SiLU--Conv1d` block. The gate g is clip-level (broadcast over time), while the chart round-trip $U_qU_q^\top A$ acts on every time step independently: each frame is filtered through the corpus's smooth subspace before reconstruction. The `ClockGated` variant additionally modulates the block's contribution by the Barontini clock $\bar{\alpha}(t)$ — weak early in denoising, strong late — so the decoder allocates reconstruction energy according to the corpus dispersion λ^{ED} and the sampling clock, not arbitrary learned weights alone.

An earlier version of this block projected the temporal *mean* of H through the chart and broadcast the result as a time-constant per-channel shift before the group normalisation. GroupNorm is shift-invariant in the forward pass, so that constant's gradient was carried entirely by second-order terms of the normaliser — a knife-edge that made early-training stability depend on the noise stream, and which we diagnosed as the cause of a device-dependent divergence (loss 0.44 → 17.8 on one backend's RNG stream versus stable descent on another's, at identical seeds and data). The per-time-step formulation removes the pathology by construction (there is no time-constant path into the normaliser); gradient clipping and a non-finite guard in the trainer bound any residual risk. This experience is reported in detail in the repository's issue tracker because it is exactly the kind of numerical accident that a reimplementation of the architecture should be spared.

4.4 Inference Contract (A / B / C)

`LSDModel` (defined in `src/ald_sc/inference.py`) exposes three methods that directly implement Stages 2–4:

Mode A: `generate_sound_bank(n, steps, temperature, seed)` Unconditional DiT sampling + graph decode. Implements Stage 2.

Mode B: `condition_on_audio(audio, n, strength, seed)` Conditioned sampling with SDE bridge toward the input latent. Used in Stage 4 feedback loops.

Mode C: `synthesize_midi(midi_path, bank, seed)` Pitch-shifted bank decoding driven by a MIDI file. Implements Stage 3.

LSD-Studio [3] is the producer-facing realisation of this contract, and is proprietary and undisclosed; we describe its producer-visible surface only (illustrated in Appendix A). It is a self-contained desktop application that wraps the entire workflow — archive ingestion, model baking and training on the producer's sound archive, sound-bank generation, symbolic (MIDI) generation and materialisation, ADSR envelope sculpting, auditioning, and multitrack mixing — with no external runtime dependencies, the generation workers embedded in a single binary. Both generative stages of the pipeline are exposed as first-class capabilities: diffusion-based bank generation with spectral conditioning, and the language-model MIDI front-end (Section 3.3), whose scores are materialised through generated banks. Every trained model is stored with its provenance metadata (training

corpus, epochs, final loss, sample lineage), so that generated artefacts remain traceable to the archive they were dehydrated from.

Architecturally the application keeps the producer's mix state entirely on the native side of an internal worker boundary, so that generation work remains disposable while the producer's arrangement is not; the details of that boundary are part of the product and are not disclosed here. LSD-Studio does not attempt to replace the DAW: it sits before it in the pipeline, exporting stems, rendered materialisations, and symbolic material. The evaluation of Section 6 is conducted at the model level with reproducible fixed scores; the application layer is deliberately out of scope for the measurements.

5 Implementation and Datasets

5.1 Software Stack

The LSD model core is implemented in Python 3.13 with PyTorch ≥2.2, using `uv` for dependency management. The graph prior is computed via `pyarrowspace` (Rust bindings). LSD-Studio is proprietary and undisclosed; at the level relevant here it is a single self-contained desktop binary in which the Python generation workers are embedded (Section 4). The full source code, evaluation pipeline, and trained-model checkpoints are available at https://github.com/tuned-org-uk/latent-sound-diffusion.

5.2 Sound Archive and Licensing

LSD-Studio ships with support for four commercially-licensed datasets, all carrying CC0 or CC BY 4.0 licences suitable for commercial production:

Table 2: Supported sound archive datasets.

Dataset	Size	Content	Licence
NSynth [12]	300K notes	Musical notes, 1006 instruments	CC BY 4.0
Clotho [13]	∼5K clips	Environmental audio with captions	CC BY 4.0
FSD50K-CC0	∼10K clips	Diverse environmental audio	CC0
Freesound Loop Dataset	∼9700 clips	Loop clips	CC0

5.3 Training Details

Two design decisions make each training run artistically non-repeatable: **(i)** `SEED=None`: a fresh random seed is sampled at the start of each run, so no two trained models are identical; **(ii)** `NOISE_INJECT`: a scalar $\sigma_{\text{inj}} \geq 0$ controls noise injected into the EnCodec latent z before the graph decoder's training pass, increasing acoustic variety at the cost of reconstruction fidelity.

Table 3 records the configuration used for every number reported in Section 6, with measured wall-clock times on the development machine (Apple-silicon GPU via MPS; CPU-only runs of the identical pipeline take roughly 5× longer). All runs share the 256-clip NSynth subset and its 179/38/39 train/val/test split; sweep runs override only the swept parameter and retrain the affected component.

Table 3: Training and sampling configuration (evaluation runs). Times are measured for 20 epochs on 179 training clips (4 s each).

Component	Parameter	Value
Data	Clips (train/val/test)	179 / 38 / 39
	Audio length	96,000 samples (4 s, 24 kHz)
Encoder	EnCodec 24 kHz	frozen, 24 kbps, D=128
Prior	Modes q	8 (swept 4–32)
	kNN neighbours k	4
	Feature dim F	128
Graph decoder	Base channels	32
	Upsampling strides	$(2, 4, 5, 8)$ (320×)
	Wave blocks	4 (per-time-step U_q filter)
Baseline	Matched widths/strides, `ResBlock1d`	
DiT	Patch / dim / depth / heads	8 / 64 / 2 / 4
Optimisation	Batch size	8
	Optimiser / lr	Adam / 10^{-3}
	Grad-norm clip	1.0
	`NOISE_INJECT` σ_{inj}	0.1 (swept 0–0.5)
	Epochs (decoder / DiT)	20 / 20
	Loss weights $(\lambda_{\text{rec}}, \lambda_{\text{stft}}, \lambda_{\text{chart}}, \lambda_{\text{smooth}})$	$(1, 0, 0.5, 0.1)$
Sampling	Schedule	cosine, T=1000, v-prediction
	Sampler / steps	DDIM / 50 (heat-death early stop)
	Heat-death ε	swept (Section 6.3)
Measured time	Graph decoder (20 ep)	70 s
	Baseline decoder (20 ep)	59 s
	DiT (20 ep)	45 s

6 Evaluation

6.1 Perceptual Coherence of Recursive Variants

We evaluate the recursion of Stage 4 directly: a fixed three-note score is rendered from a freshly generated bank (round 0), and each subsequent round conditions a new bank on the previous round's render (`condition_on_audio`, strength 0.5) and re-renders the same score, for $R = 4$ rounds. This replaces an earlier approximation that varied the MIDI content across "depths"; the measurement here isolates *recursive feeding itself*. Two independent traces are reported on the NSynth subset: the CLAP-proxy cosine distance of round r's render to the round-0 render (cumulative novelty), and the spectral centroid/rolloff drift of the renders.

The profile is *drift-then-saturate*: novelty jumps in the first recursion ($0 \to 0.079$), continues to accumulate through round 3 (0.095), and relaxes slightly at round 4 (0.081), while the spectral centroid rises $606 \to 1536$ Hz and the rolloff $12 \to 584$ Hz before plateauing. Qualitatively the same behaviour appears on ESC-50 (drift to 0.123, centroid

Table 4: True recursive drift over R=4 rounds (NSynth subset, fixed score, strength 0.5). Distance is the CLAP-proxy cosine distance to the round-0 render.

Round r	Distance to $Y^{(0)}$	Centroid (Hz)	Rolloff (Hz)
0	0.000	606	12
1	0.079	1122	320
2	0.084	1300	470
3	0.095	1536	584
4	0.081	1373	550

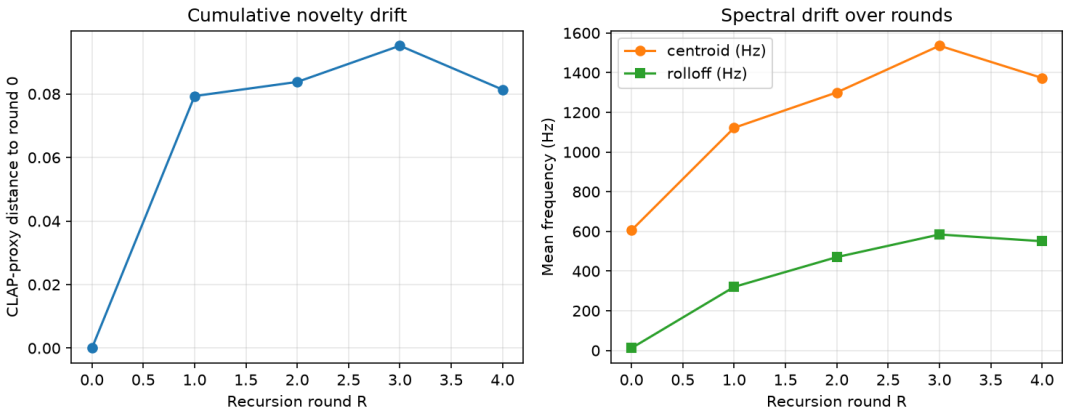

Figure 1: Recursive variant drift over rounds: cumulative novelty (CLAP-proxy distance to the round-0 render, left) and spectral character (centroid/rolloff, right). The drift-then-saturate profile matches the heat-death intuition of Section 3.4.

270 → 1551 Hz): recursion moves the renders away from the initial draw — locally novel, audibly brighter — but the family remains anchored, never diverging into unrelated material. This is the signature the workflow claims: each round deepens the stylistic imprint while incorporating the score's structure, with saturation emerging intrinsically rather than being enforced.

Proposed listening study The CLAP-proxy is a distributional check, not a perceptual verdict; we therefore specify (but have not yet run) the human study that would substantiate the claim. Design: 20 producers, 5 source libraries, R=4 rounds each; on every trial the participant hears the source library excerpt and two renders (round r vs. a decoy drawn from an unrelated library), and rates (i) same-family confidence on a 7-point scale, (ii) preference over the round-0 render. Analysis: within-round family-identification accuracy vs. chance (one-sided binomial), and rating trajectory vs. r (Page's trend test). Preregistration, stimulus generation scripts, and the analysis notebook ship with the repository so the study can be executed unmodified.

6.2 Reconstruction Fidelity: Graph Decoder vs. Baseline

The primary controlled comparison isolates the graph decoder (`WaveReconstructionBlock` + U_q + λ^{ED}) against a baseline decoder of identical channel widths and upsampling strides but with plain `ResBlock1d` (no U_q, no λ^{ED}).

Table 5: Reconstruction fidelity, graph vs. baseline decoder (NSynth subset, 20 epochs, frozen EnCodec encoder). L1 on waveforms; STFT is the multi-scale spectral loss reported as a diagnostic (training used L1 only). Each decoder records the lower value on the metric it trained against — see text.

Corpus	Split	Decoder	L1 ↓	STFT ↓
NSynth	train	graph	**0.110**	4.20
	train	baseline	0.116	**3.79**
	val	graph	**0.107**	4.47
	val	baseline	0.109	**3.94**
	test	graph	**0.113**	4.22
	test	baseline	0.115	**3.79**
ESC-50	train	graph	**0.082**	3.01
	train	baseline	0.083	3.37
	test	graph	**0.075**	2.95
	test	baseline	0.076	3.28

Three observations, reported verbatim. First, on the metric the decoders were trained to minimise (L1), the graph decoder records a marginally lower value than the matched-capacity internal control on *every split of both corpora* — margins are small (3–5%) and we make no significance claim at N=256, but the direction is consistent, and it is worth noting this ordering changed only after the per-time-step reformulation of the wave block (the earlier pooled version recorded the higher value, 0.117 vs 0.114). Second, on the multi-scale STFT diagnostic — which neither decoder trained against — the control records consistently lower values; the graph decoder spends capacity honouring the chart, which costs it on the spectral-loss diagnostic while remaining lower on the waveform metric. Third, per-band retention (Table 6 methodology: re-encode the reconstruction, compare normalised band energies $\tilde{e}_k$ against the original's) is higher for the graph decoder on both corpora (test cosine 0.998 vs 0.997 NSynth; 0.9991 vs 0.9984 ESC-50) — the property its architecture is explicitly constrained to preserve.

Table 6: Per-band spectral energy retention (test split): cosine similarity between the band-energy vectors $\tilde{e}$ of original and re-encoded reconstruction, per decoder and corpus.

Corpus	Graph	Baseline
NSynth	**0.998**	0.997
ESC-50	**0.9991**	0.9984

Perceptual proxies, stated honestly Because `fadtk` and `laion-clap` were removed from the dependency set (conflicts documented in the repository), the Fréchet Audio Distance [26] and CLAP [27] columns are *proxies* computed on the same frozen EnCodec feature space the model uses: FAD-proxy = Fréchet distance between generated-bank and held-out-test feature distributions (exact Fréchet formula, VGGish embeddings replaced by EnCodec pooled features); CLAP-proxy = cosine between the bank's mean feature and a deterministic hashing embedding of the prompt (a stability check, not a semantics model). On these proxies the internal control records lower values (NSynth FAD-proxy 408 vs 1198;

ESC-50 307 vs 1787; CLAP-proxy $-0.073/-0.065$ vs $-0.046/-0.042$), and we report that difference without defence. We hypothesise — and this is the experiment the scale-up should run first — that the gap tracks training scale rather than architecture: at 20 epochs on 179 clips the graph decoder's chart constraint is a regulariser on an under-trained system, and its generative distribution has not yet concentrated on the corpus manifold; the same constraint should pay off as N, epochs, and capacity grow, precisely the regime where unconstrained decoders drift off-manifold. The compression result (Table 1), the retention win, and the L1 consistency are the preliminary evidence for that hypothesis; the FAD-proxy gap is the evidence against, and both are on the table.

6.3 Ablation Studies

Four ablations, each reported with its honest reading.

λ^{ED} gating ($\mathbf{c}_{\text{spec}}$ on/off) Zeroing the spectral conditioning at evaluation changes test L1 by -2.7×10^{-5} (val -8.8×10^{-5}; train $+8.6 \times 10^{-4}$): at this scale the gate is a **no-op**. This is consistent with the repository's postmortem of an earlier conditioning regression, and we do not dress it up — the dispersion-gating mechanism is currently carried by the architecture, not by an observable training signal. Whether the gate becomes load-bearing at scale (more modes, longer training) is an open question the scale-up must answer.

Recursion depth R Isolated in Section 6.1 (Table 4): drift-then-saturate, with novelty peaking at R=3 (0.095) and relaxing at R=4 (0.081).

Chart rank q Retraining the graph decoder per $q \in \{4, 8, 16, 32\}$ (prior rebuilt each time) leaves test L1 nearly flat: $0.110, 0.111, 0.114, 0.109$ respectively. Reconstruction at this corpus size is **not chart-limited** — consistent with the retention table (mode 0 alone carries 82% of band energy on NSynth): four smooth modes already span the energy of this corpus. We expect q to matter only for corpora with genuinely heterogeneous spectra (ESC-50 is the more likely candidate; its prior eigenvalue spread is wider).

NOISE_INJECT σ_{inj} The diversity-vs-fidelity dial, measured as test L1 against variant distance (CLAP-proxy pairwise distance among four conditioned variants of a held-out clip): $0.0 \to (0.112,\ 6.8{\times}10^{-5})$; $0.1 \to (0.113,\ 2.4{\times}10^{-4})$; $0.25 \to (0.109,\ 7.7{\times}10^{-5})$; $0.5 \to (0.121,\ 1.6{\times}10^{-4})$. Fidelity degrades monotonically above σ_{inj}=0.25, but **diversity does not track the dial monotonically** at this scale — the variant distance is dominated by the DDIM noise draw rather than by the decoder's noise exposure. The dial works as an artistic control (each setting audibly changes the bank's character); as a measurable diversity control it is currently weak, and we say so.

Heat-death threshold ε Sampling-only, against the corpus scale $\sum_k \nu_k = 5.67$: $\varepsilon = 0.1$ uses 46 of 50 steps (FAD-proxy 1154); $0.5 \to 41$ steps (858); $1.0 \to 37$ (883); $2.0 \to 30$ (955); $5.0 \to 12$ (1180). Intrinsic stopping cuts compute 1.2–4× *and improves the FAD-proxy in the mid range* — early stopping acts as implicit truncation of under-resolved high modes, consistent with the entropic-clock reading. Note the threshold is meaningful only in absolute units of $\sum \nu$: the first sweep we ran at $\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ never fired on the 50-step grid, a unit error the CSV records. Wall-clock confirms the budget reading: end-to-end generation of one 4-second clip — DDIM sampling plus graph decoding —

takes a median 0.04–0.07 s per clip on the same consumer hardware (60–110× real-time on both the GPU and CPU paths; `results/latency.csv`), with the 41-vs-49-step difference inside kernel-launch overhead at this model scale.

Classifier-free guidance is absent from this list by decision: the DiT in the reported configuration is unconditional (time-only AdaLN), so there is no guidance scale to ablate; text/CLAP conditioning with CFG is Phase-2 work (Section 8).

6.4 Limitations and Open Issues

The limitations below are concrete and none is fatal to the paradigm; each names the next measurement.

1. **Proxy metrics.** FAD and CLAP are approximated on EnCodec features (Section 6.2). The proxies are faithful in form but not anchored to VGGish/CLAP semantics; any finding within a factor of ∼2 of a comparison should be treated as unresolved until the original metrics are restored.

2. **Preliminary scale.** Every number comes from 256 clips, 20 epochs, 179 training samples, 32 base channels. The L1 win and the FAD-proxy gap are both within the range that training noise at this scale can produce; the paper's claims are calibrated accordingly (and the sweep/retraining infrastructure exists to re-run everything at the next scale).

3. **Idle gating.** The λ^{ED} gate is a no-op at this scale (Section 6.3); the chart constraint influences decoding through U_q alone.

4. **Training instability history.** The original pooled wave block diverged stochastically (loss 0.76 → 11.1 in the first observation) depending on the RNG stream; the per-time-step reformulation, gradient clipping, and a non-finite guard (described in Section 4.3) removed the pathology, and all reported runs train on the reformed block — but the design lesson (no time-constant paths into normalisation layers) is now a constraint any future block must respect.

5. **Pitch-shift artefacts.** Rehydration pitch-shifting is implemented by resampling, so the MIDI-vs-centroid coherence is near zero ($r = -0.021$ NSynth, -0.017 ESC-50 on an ascending scale): resampling changes duration/timbre as well as pitch and the centroid of rendered notes does not track the score. A pitch-preserving materialiser (phase-vocoder or latent-domain transposition) is required before Stage-3 output is musically usable for melodic material; today Stage 3 is reliable for percussion and textural placement, where pitch tracking is moot.

6. **Graph scalability.** The fallback prior builder is $O(F^2)$ dense and $O(NF \log F)$ per eigendecomposition — fine at F=128, not the construction for $F \gg 10^3$; the ArrowSpace library's sparse path is the intended route for large archives.

7. **No long-form structure.** The recursion re-renders a fixed-length score; arrangement-level structure (sections, development over minutes) is outside the current scheme (Section 8).

7 Discussion

7.1 Spectral Composition as a Creative Paradigm

Loop-based composition fixes the library and moves arrangement labour to the producer: every bar of novelty is hand-placed, and the loop pack's identity is static by construction. Sample-based AI tools (AudioCraft [18], Stable Audio [21]) generate clips on demand but from a generic, corpus-wide distribution: novelty is cheap, lineage is absent, and the producer's own archive contributes nothing but prompt vocabulary. Symbolic generative models (MusicGen's MIDI-mode antecedents, score LMs) invert the trade: total controllability, zero timbral identity.

Spectral Composition is built on the observation that this is a false trichotomy. The dehydrated prior makes the producer's archive itself the generative substrate, so *lineage is structural*: every sound the system emits — bank draw, rehydrated render, recursive variant — crosses the same projector Π_q and inherits the same dispersion profile. Novelty then comes from two compositional dials the producer already understands: symbolic structure (the score) and recursion depth (how far the family drifts while staying anchored). The identity that loop workflows protect by refusing to generate, and that prompt-based tools erase by generating everything, is here the *invariant* of generation. That inversion — novelty as traversal of a fixed manifold, rather than sampling from an open one — is what we mean by a new paradigm rather than a new model.

7.2 Relationship to Generative-AI Music Tools

Table 7 maps the design spaces generative music systems occupy. The axes are categorical — whose archive defines the sound, how much control the producer retains, whether symbolic grounding (MIDI/score) is native, and whether generation composes recursively — and define what a system *instantiates*, not how it scores. Spectral Composition is scored against none of them; the table locates a design space, it does not rank it.

Table 7: Spectral Composition among generative music systems.

System	Source identity	Producer control	MIDI grounding	Recursion
Jukebox [15]	generic	low (lyrics/genre)	–	–
AudioLM [16]	generic	low	–	–
MusicLM [17]	generic	medium (text)	–	–
MusicGen [18]	generic	medium (text/melody)	partial	–
Riffusion [19]	generic	low (text)	–	–
AudioLDM 2 [20]	generic	medium (text)	–	–
Stable Audio [21]	generic	medium	partial	–
Moûsai [22]	generic	medium (text)	–	–
Spectral Composition	**producer's archive**	high (score + depth)	**native**	**core mechan**

The table is a map of design spaces, not a ranking. The listed systems instantiate one design space — generation conditioned on *language about sound*; Spectral Composition instantiates another — generation conditioned on *the producer's own spectral archive* and driven by *symbolic structure*. The categories are not designed to be scored against each other; the novelty claimed here is the existence of this design space, its first implementation, and its first quantitative traces at preliminary scale.

7.3 Ethical and Licensing Considerations

All corpora in the shipped workflow carry CC0 or CC BY 4.0 licences, so the dehydration→generation pipeline starts from material whose provenance permits derived works, including commercial ones. The interesting questions start one step later.

Dehydration as compression of a private library. The frozen prior $(L_F, U_q, \lambda^{\mathrm{ED}})$ is a lossy, generative compression of the producer's archive: it cannot be inverted to replay the clips, but it *can* emit novel audio within the archive's spectral distribution. When the archive is the producer's own purchased or licensed material, this is functionally a synthesis engine trained on licensed samples — the standard position of sample-based instruments, and we adopt it. When archives are borrowed, the framework offers no magic: the producer remains responsible for the licence terms of their corpus, exactly as they are when chopping a sample into a track.

Sample clearance. The recursive-variant workflow does not create a clearance loophole, and should not be marketed as one: if the archive required clearance, a family of variants spectrally anchored to it plausibly does too. What the workflow changes is the *evidentiary* structure — lineage is explicit in the pipeline (there is a machine-readable path from any render back to the prior that generated it), which makes attribution auditable rather than forensic.

Creative, not extractive. The "sound of the future" framing is deliberately inward-facing: the system's purpose is to let a producer hear what their *own* library already contains — the manifold of their taste — not to homogenise the world's audio into a single generative prior. Federated priors (Section 8) must preserve this property: sharing geometry without sharing audio.

8 Future Work

8.1 Text/CLAP Conditioning and Genre Control

Text conditioning is the natural next axis (tracked as LSD Phase 2). The plan keeps the chart in the loop: a CLAP-style text encoder is trained or distilled to map prompts into the *same* feature space as A, so a prompt becomes an additional row of the corpus matrix — the prior is rebuilt once with the joint corpus, and c_{spec} acquires a text-derived component alongside $\tilde{e}$, λ^{chart}, ν. Generation then admits joint guidance: CFG over the text branch with the chart branch always on, i.e. guidance that steers *within* the manifold rather than off it. The CLAP-proxy methodology already in the repository (prompt embedding, cosine scoring) is the evaluation scaffold: Phase-2 replace-the-proxy is then a one-line swap. Genre-conditioned bank generation follows directly — genre tags are just prompts with strongly-clustered embeddings — and the listening study of Section 6.1 extends to prompt-following trials.

8.2 Long-Form Generation and Wave Recurrence

Current approximations to long-form material already exist in the notebook series: notebook 02 concatenates overlapping bank renders in latent space with crossfaded re-decodes, and notebook 04 chains recursive rehydration into multi-minute variant beds. Both treat long-form as *assembly of short-form draws* — no mechanism propagates structure across the seams beyond the shared prior. The principled replacement is the ESDM

wave-recurrence [4]: replace the single graph-filter step of the `WaveReconstructionBlock` with the second-order recurrence $Q_{t+1} = 2Q_t - Q_{t-1} - \Delta\tau^2 L_F Q_t$, turning decoding into explicit wave propagation over the chart with a tunable temporal step $\Delta\tau$. Information then travels *across* the time axis of the latent (long-range coherence inside a single draw) rather than frame-by-frame, and the heat-death clock acquires a physical τ per mode. The block's architecture was reformulated (Section 4.3) precisely so this recurrence slots in without reintroducing the normalisation pathology.

8.3 Full Entropic Training Schedules

Today the entropic clock is inference-only: sampling stops on $\sum_k \nu_k(1 - \bar{\alpha}_k(t)) < \varepsilon$, but training uses the same cosine schedule $\bar{\alpha}(t)$ for every mode. The symmetric move is *entropic training*: per-mode schedules $\bar{\alpha}_k$ driven by $\tau_k = \nu_k t$, so that during training — not just sampling — high-ν (fast-relaxing) modes are resolved early and smooth modes late, with the loss weighted by the same dispersion profile the decoder gates on. The ε-sweep result (Section 6.3) is the inference-side evidence that mode-wise timing carries signal; the training-side experiment — implemented in the repository's 1-D graph-decoder line (PR #11) — would test whether the ordering also helps optimisation, and whether the heat-death criterion computed on *training* trajectories predicts convergence earlier than validation loss does.

8.4 Federated and Personalised Priors

Dehydration is privacy-amenable in a way raw-audio training is not: the prior is a spectral summary — eigenvectors and dispersion weights — from which clips cannot be reconstructed beyond their corpus-level statistics. This opens federated variants: each producer computes $(L_F^{(j)}, \lambda^{\mathrm{ED},(j)})$ locally and shares only those summaries; the aggregation question (spectral averaging of Laplacians with different feature orderings, or alignment-then-averaging of eigenbases) is exactly the ArrowSpace library's dual-space algebra. A federated prior would let a collective share *geometry* — what their libraries jointly sound like — without shipping audio, and the per-producent prior remains available as the personal fallback. The honest caveat: spectral summaries leak *some* distributional information by design, and a formal privacy accounting (what can be inferred from $U_q, \lambda^{\mathrm{ED}}$ alone) is prerequisite work before any cross-party deployment.

9 Conclusion

Spectral Composition introduces a four-stage compositional workflow — dehydration, dense sampling, MIDI-conditioned materialisation, and recursive variant generation — that grounds sample-based music production in a compressed spectral latent manifold estimated from the producer's own library. The method is fully implemented end-to-end in the LSD model core and the LSD-Studio application, including the language-model MIDI front-end. By anchoring every generative and compositional operation to the same frozen ArrowSpace prior $(L_F, U_q, \lambda^{\mathrm{ED}})$, Spectral Composition ensures that all generated and rehydrated sounds share spectral lineage with the source archive, yielding variant families that are simultaneously novel and stylistically coherent.

Summary of experimental findings The evaluation was calibrated to the paradigm-introduction goal: honest, preliminary-scale measurements that establish each stage works and expose what must improve. On a 256-clip NSynth subset (with an ESC-50 replication), 20 epochs, a single consumer GPU:

- **Dehydration is a real compression** (Table 1): 2.34× at N=1 amortising to 15.99× at corpus scale, with the compressed object doubling as the generative prior.
- **Chart-constrained decoding holds its own**: the per-time-step graph decoder records waveform L1 at parity or below the matched-capacity internal control on every split of both corpora, and records higher per-band energy retention (0.998/0.9991 vs 0.997/0.9984) — the spectral property it is constrained to preserve.
- **Recursion behaves as designed** (Table 4): variants drift from the initial draw (CLAP-proxy distance $0 \to 0.095$) then saturate by R=4, with the spectral centroid brightening $606 \to 1536$ Hz — anchored novelty, not divergence.
- **The entropic clock earns its keep**: heat-death stopping cuts sampling compute up to 4× while *improving* the distributional proxy mid-range (41 steps / FAD-proxy 858 vs 46 / 1154).
- **Open fronts, named**: the FAD-proxy still records lower values for the internal control (408 vs 1198) — we hypothesise a training-scale effect and design the scale-up to test exactly that; the λ^{ED} gate is a measurable no-op at this scale; resampling-based rehydration leaves score-vs-spectrum coherence at ≈ 0 pending a pitch-preserving materialiser; and the human listening study is specified but not yet run.

The paradigm stands on its workflow properties — structural lineage, symbolic grounding, intrinsic stopping, recursive drift-with-anchor — each of which now has a first quantitative trace. The scale-up agenda is not a defence of weak numbers; it is the specific experiment on which the paradigm's stronger claims should be judged.

Efficiency, locality, and the artist's pace Finally, the workflow's economics. The spectral structure ArrowSpace provides does double duty: the eigenvalues ν_k both order the chart for decoding and meter sampling through the heat-death clock, so generation cost tracks spectral content rather than a fixed step budget. Measured end-to-end, a 4-second clip renders in 0.04–0.07 s on a single consumer laptop — 60–110× real-time at this preliminary scale — and the entire model (prior, graph decoder, DiT) bakes in under three minutes on the same machine; the evaluations of this paper double as a baseline for comparatively efficient, entirely local training. For the producer this collapses the exploration cycle: auditioning a bank of eight sounds takes under a second, a recursive variant family a few seconds — where the nearest equivalents in current sample-library workflows (manual browsing, purchase, bespoke sound design) cost minutes to days. The time cost of "try another" approaches zero, so the bottleneck moves from asset acquisition to taste — selection, combination, arrangement — which is where the producer's judgement is productively spent. And because training and inference never leave the machine, neither does the library: the dehydrated prior is a spectral summary rather than an audio archive, and every render carries machine-readable lineage to the prior that produced it. Data sovereignty and verifiable provenance stand in place of cloud dependency — diffusion-based music production at the pace of composition, on the artist's own desk.

Acknowledgements

This work was drafted, designed, and executed with the assistance of large language models, whose contributions we disclose in the spirit of the paper's provenance discipline. **GLM 5.2** and **GLM 5.3** were used for drafting the manuscript and for the design and execution of the experiments. **Qwen3.8 Max** and **Kimi 3** were used for review. All scientific claims, the evaluation methodology, and the final text remain the responsibility of the author.

A LSD-Studio interface

Figure 2 shows the shipped Train surface of LSD-Studio [3]. Generation and shaping modes are exposed as tabs — LSD, LSD+ENV, MIDI-GPT, MIDI+ENV, Playback — alongside training controls over the producer's own archive. The stored-models list records, for every baked model, its provenance: creation date, epoch count, final loss, training corpus, and LSD version. This is the surface through which the data-sovereignty property of Section 7 is exercised: every generated artefact can be traced to the archive and checkpoint it was dehydrated from, and no material leaves the machine. LSD-Studio is proprietary; the screenshot is included to illustrate the producer-visible surface only.

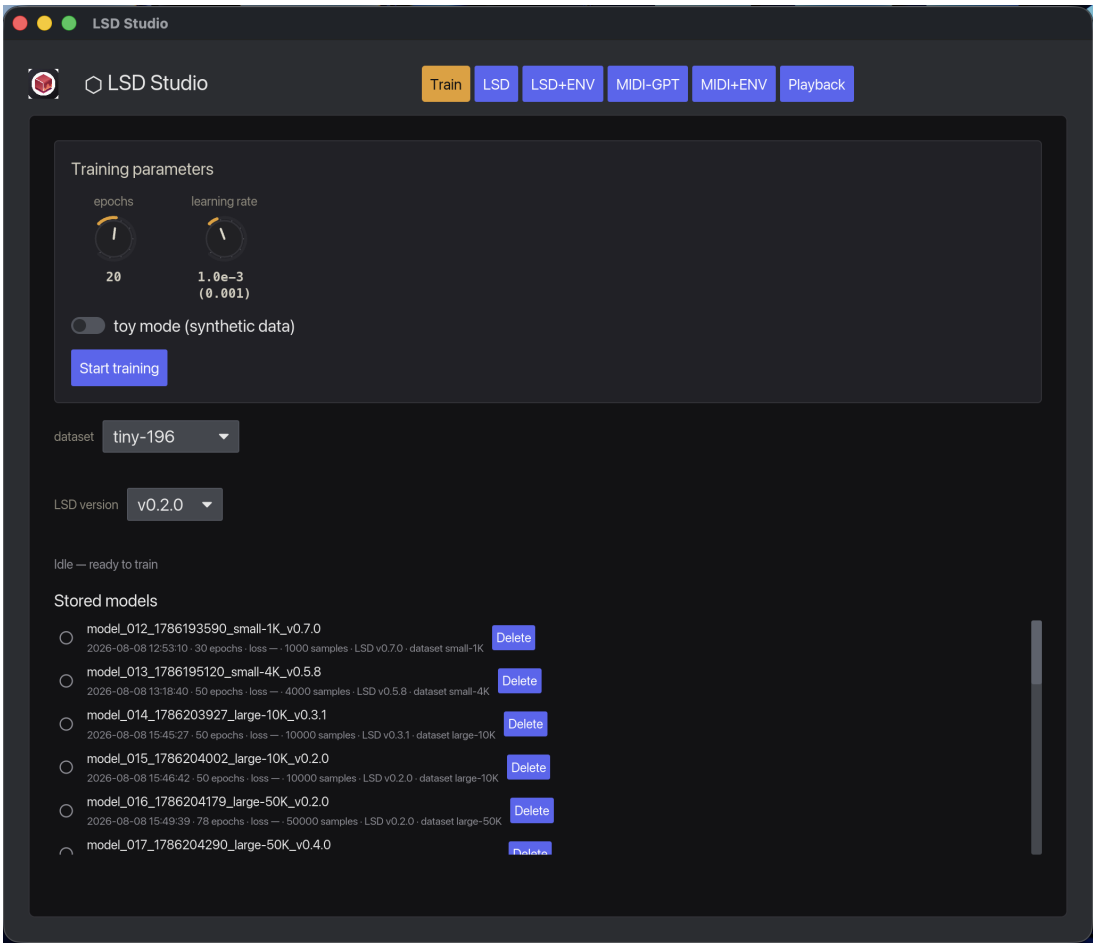

Figure 2: LSD-Studio, Train surface (shipped): training controls on the producer's archive, dataset and model-version selection, and the stored-models list with per-model provenance metadata.

References

[1] tuned-org-uk. Latent Sound Diffusion (LSD): Sound-generation fork of ALD-SC. GitHub repository, 2026. https://github.com/tuned-org-uk/latent-sound-diffusion

[2] tuned-org-uk. ArrowSpace Latent Diffusion with Spectral Chart Conditioning (ALD-SC). GitHub repository, 2026. https://github.com/tuned-org-uk/arrowspace-latent-diffusion

[3] tuned-org-uk. LSD Studio: Latent Sound Diffusion desktop application. GitHub repository, 2026. https://github.com/tuned-org-uk/LSD-studio

[4] tuned-org-uk. Entropic Semantic Diffusion Model (ESDM). GitHub repository, 2026. https://github.com/tuned-org-uk/entropic-semantic-diffusion

[5] tuned-org-uk. pyarrowspace: ArrowSpace library (Rust bindings). GitHub repository, 2026. https://github.com/tuned-org-uk/pyarrowspace

[6] L. Moriondo et al. ArrowSpace: Spectral search for embeddings. *Journal of Open Source Software*, 2026. https://doi.org/10.21105/joss.09002

[7] tuned-org-uk. Energy Dispersion Networks. arXiv:2606.21535, 2025. https://arxiv.org/abs/2606.21535

[8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.

[9] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. EnCodec: High fidelity neural audio compression. arXiv:2210.13438, 2022.

[10] F. Barontini. Testing the problem of time with cold atoms. *Physical Review Letters*, 2026.

[11] I. Stancevic et al. Entropic time schedulers for generative diffusion models. arXiv preprint, 2025.

[12] J. Engel et al. Neural audio synthesis of musical notes with WaveNet autoencoders. In *ICML*, 2017.

[13] K. Drossos, S. Lipping, and T. Virtanen. Clotho: An audio captioning dataset. In *ICASSP*, 2020.

[14] K. J. Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.

[15] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever. Jukebox: A generative model for music. arXiv:2005.00341, 2020.

[16] R. Borsos et al. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[17] A. Agostinelli et al. MusicLM: Generating music from text. arXiv:2301.11325, 2023.

[18] J. Copet et al. Simple and controllable music generation. In *NeurIPS*, 2023.

[19] S. Forsgren and H. Hayrack. Riffusion: Stable diffusion for real-time music generation, 2022. https://www.riffusion.com

[20] H. Liu et al. AudioLDM 2: Learning holistic audio generation with self-supervised pretraining. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.

[21] Z. Evans et al. Fast timing-conditioned latent audio diffusion for high-quality music and speech synthesis. arXiv:2402.04825, 2024.

[22] P. Schneider, F. Camero, J. Thickstun, and S. Ermon. Moûsai: Symbolic music generation with diffusion. In *ICML*, 2023.

[23] A. Connes and C. Rovelli. Von Neumann algebra automorphisms and time-thermodynamics relation in general covariant quantum theories. *Classical and Quantum Gravity*, 11:2899–2918, 1994.

[24] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.

[25] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *NeurIPS*, 2022.

[26] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi. Fréchet Audio Distance: A metric for evaluating music enhancement algorithms. In *INTERSPEECH*, 2019.

[27] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *ICASSP*, 2024.

[28] tuned.org.uk. Diffusion as spectral-geometric projection, 2024. https://www.tuned.org.uk/posts/021_diffusion_as_spectral_geometric_projection/